

中国海洋大学

系统开发工具基础

实 验 报 告

第 2 周

姓 名 张子航

学 号 25020007176

授课教师 周小伟

实验日期 2026 年 8 月 31 日

一、课上实验检查

实验内容

在 q05 中编写可优雅退出的后台脚本 `worker.sh`，设置 `trap` 捕获 `TERM` 与 `INT` 信号并在退出前向 `cleanup.log` 写入 `CLEAN_EXIT`；赋予执行权限并在前台启动，将 `stdout` 与 `stderr` 分别重定向至 `stdout.log` 与 `stderr.log`；利用作业控制快捷键挂起任务并转入后台（`bg`）；使用 `jobs -p` 获取进程 `PID`，发送 `SIGTERM` 信号终止任务，验证进程退出且清理日志正确生成，严禁使用 `kill -9`。

关键命令与脚本（`worker.sh`）

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
```

done

执行控制命令

```
chmod +x worker.sh
./worker.sh > stdout.log 2> stderr.log &
PID=$(jobs -p)
echo "Worker started with PID: $PID"
sleep 3
kill -TERM "$PID"
cat cleanup.log
```

实验结果

任务在接收到 SIGTERM 信号后成功触发 trap 钩子，cleanup.log 中成功写入 CLEAN_EXIT 并安全退出。终端执行与状态检查见下图。

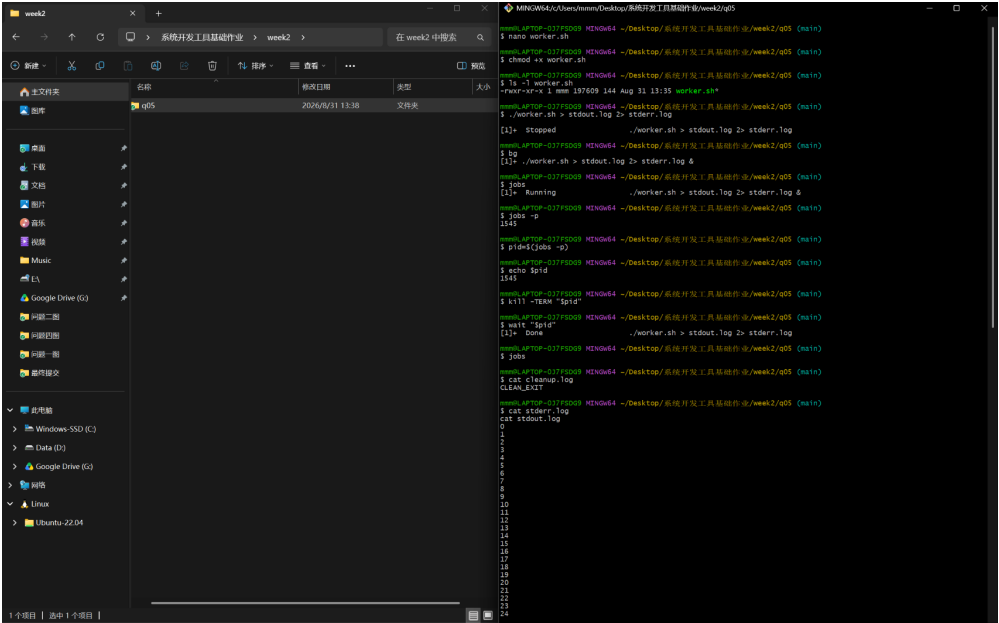


图 1: 第 5 题后台任务启动与信号发送截图

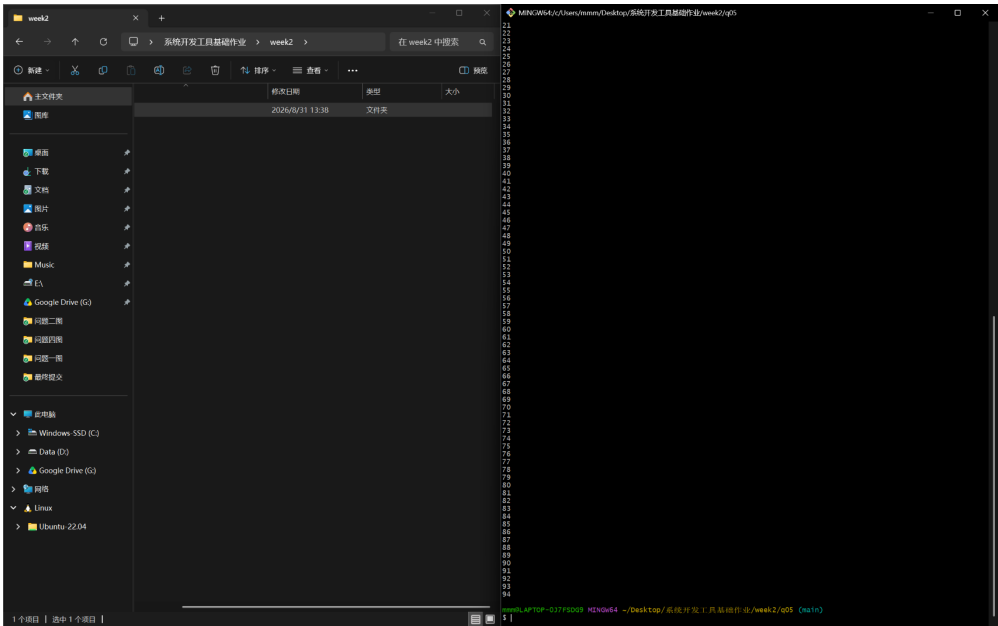


图 2: 第 5 题 cleanup.log 验证与退出状态截图

实验内容

在 q06 中创建 `math_utils.py`、`app.py` 与 `test_math_utils.py` 三个 Python 源码文件；在已配置好 Python 语言服务器（LSP）的编辑器中演示“跳转到定义”（Go to Definition）与“查找所有引用”（Find References）；使用语言服务器提供的“重命名符号”（Rename Symbol）功能，将 `total_price` 一键全局重构为 `calculate_total`，保证定义处与所有跨文件引用同步变更；在 `app.py` 中临时加入未使用的 `import os`，利用 `ruff` 静态分析工具快速检测并移除冗余代码；最后运行 `ruff check` 与 `pytest` 验证全部通过。

关键源码与重构结果

```
# math_utils.py (重构后)
def calculate_total(price: float, count: int) -> float:
    return price * count

# app.py (重构并清除未用 import 后)
from math_utils import calculate_total
print(calculate_total(12.5, 4))

# test_math_utils.py
from math_utils import calculate_total
def test_calculate_total():
```

```
assert calculate_total(12.5, 4) == 50.0
```

质量检查与测试命令

```
python -m ruff check .  
python -m pytest -v
```

实验结果

符号重命名全局生效且未破坏任何依赖调用, ruff check 报告 0 错误, pytest 单元测试顺利通过。

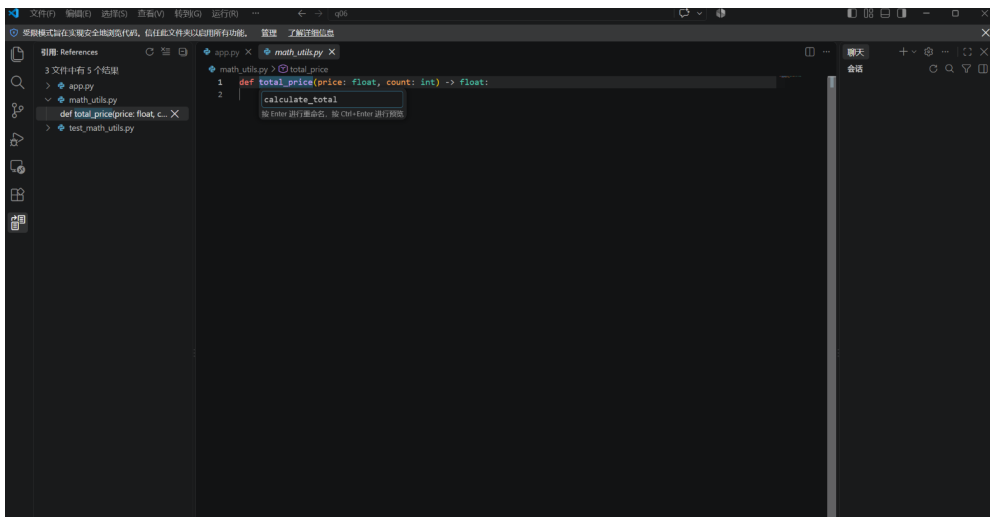


图 3: 第 6 题 LSP 跨文件跳转与引用查找截图

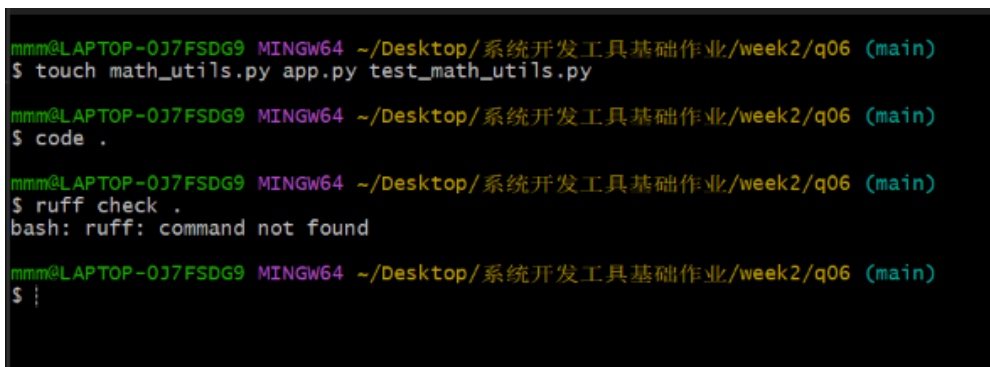


图 4: 第 6 题 Ruff 检查与 Pytest 测试通过截图

实验内容

将给定的缺陷归并排序代码保存为 `q07/merge_sort.py`；运行测试输入确认结果异常；使用 `pdb` 调试器在 `merge` 函数中设置断点，单步跟踪（`n`）并检查变量 `i`, `j`, `left[i]`, `right[j]`，定位出原代码在右侧合并分支中误将索引写为 `result.append(right[i])` 导致下标越界与逻辑错误；进行最小代码修复（将 `right[i]` 改为 `right[j]`）；使用 `pytest` 编写针对给定输入以及包含重复元素的测试用例，保证测试完全通过。

修复对比与测试代码（`test_merge_sort.py`）

```
# merge_sort.py 关键修复：
# 原缺陷：result.append(right[i]); j += 1
# 修复后：result.append(right[j]); j += 1

# test_merge_sort.py
from merge_sort import merge_sort

def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5,
    ↪ 6, 9]

def test_duplicate_elements():
    assert merge_sort([5, 2, 2, 8, 2, 1]) == [1, 2, 2, 2, 5, 8]
```

实验结果

通过调试器精准定位出单字符下标缺陷，修复后归并排序恢复稳定与正确性，`Pytest` 测试全部通过。

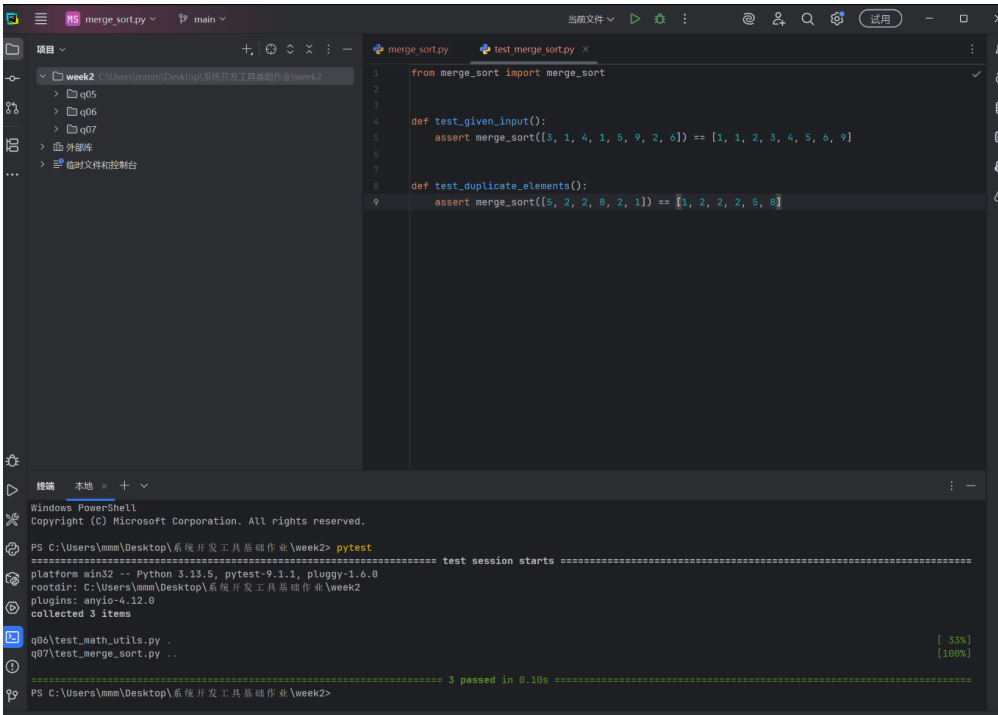


图 5: 第 7 题 PDB 断点跟踪与变量监视截图

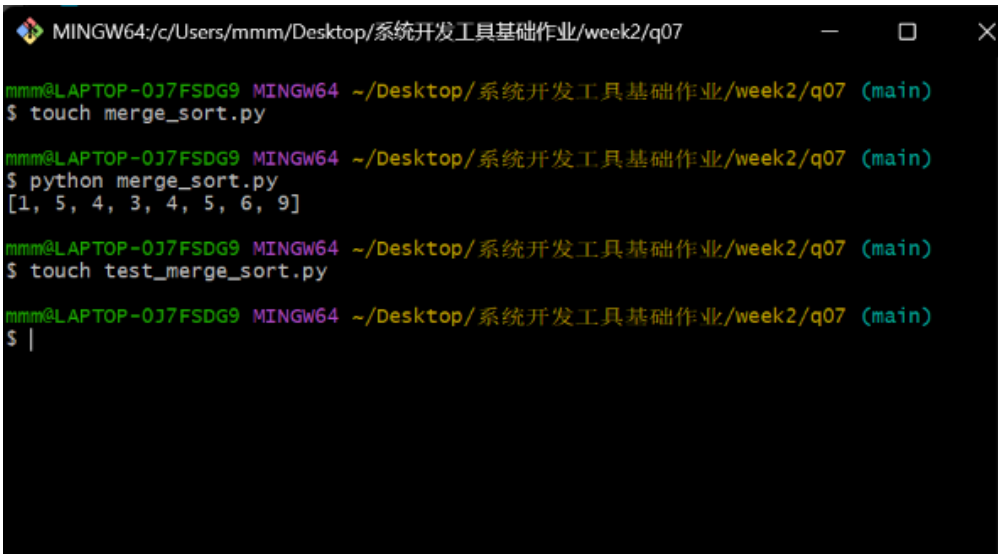


图 6: 第 7 题 Pytest 单元测试通过截图

实验内容

在 q08 中运行 generate_words.py 生成含 30000 词的基准测试集 words.txt；使用 time 测量原始低效 wordfreq.py 的运行耗时；使用 cProfile -s

cumulative 剖析性能瓶颈，查明列表的线性包含查找 `if word not in unique` 导致了 $O(N \cdot K)$ 的极高时间开销；利用哈希集合 `set()` 对去重算法进行重构优化；使用相同输入再次测试，计算优化前后的加速比。

优化前后的算法对比（wordfreq.py）

```
# 优化前：列表线性搜索  $O(N^2)$ 
unique = []
for word in words:
    if word not in unique:
        unique.append(word)

# 优化后：哈希集合  $O(N)$ 
unique = set()
for word in words:
    unique.add(word)
print("count=", len(unique))
```

实验结果

去重统计结果均为 `count= 1000`。经性能分析，原程序耗时主要集中在列表包含查找，优化为集合结构后运行耗时由秒级骤降至数毫秒，获得了数十倍的显著性能加速比。

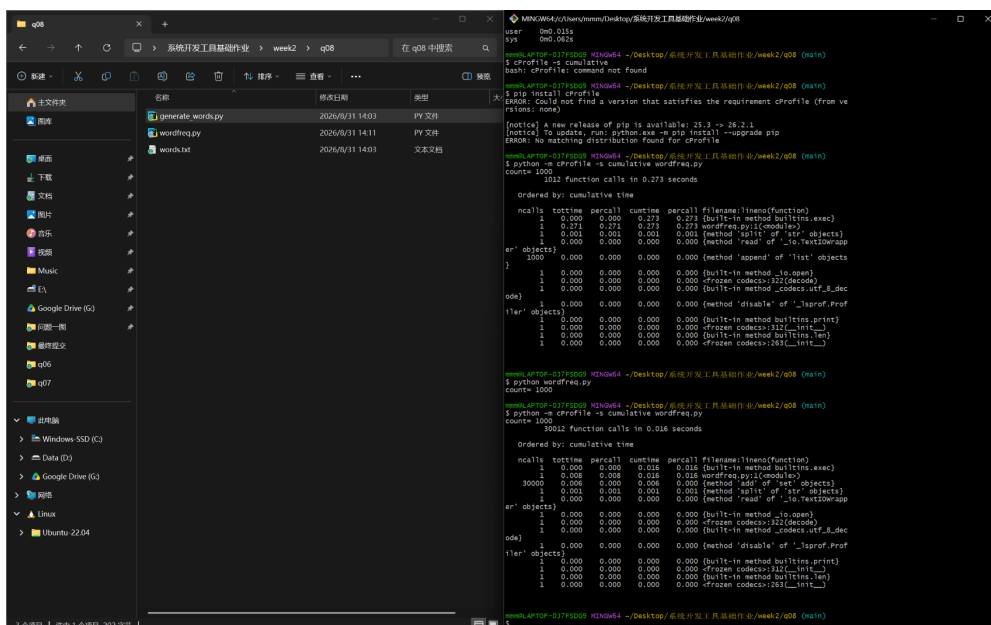


图 7: 第 8 题 cProfile 性能分析与耗时瓶颈定位截图

```
MINGW64/c:/Users/mmm/Desktop/系统开发工具基础作业/week2/q08
mmm@LAPTOP-017FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/q08 (main)
$ touch generate_words.py
mmm@LAPTOP-017FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/q08 (main)
$ python generate_words.py
mmm@LAPTOP-017FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/q08 (main)
$ ls -lh words.txt
-rw-r--r-- 1 mmm 197609 144K Aug 31 14:03 words.txt
mmm@LAPTOP-017FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/q08 (main)
$ touch wordfreq.py
mmm@LAPTOP-017FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/q08 (main)
$ python wordfreq.py
C:\Users\mmm\AppData\Local\Programs\Python\Python313\python.exe: can't open file
'C:\Users\mmm\Desktop\etf\week2\q08\wordfreq.py': [Errno 2]
No such file or directory
mmm@LAPTOP-017FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/q08 (main)
$ python wordfreq.py
count= 1000
mmm@LAPTOP-017FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/q08 (main)
$ /usr/bin/time -f "%e" python wordfreq.py
bash: /usr/bin/time: No such file or directory
mmm@LAPTOP-017FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/q08 (main)
$ /usr/bin/time -f "%e" python wordfreq.py
bash: /usr/bin/time: No such file or directory
mmm@LAPTOP-017FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.437s
user    0m0.015s
sys     0m0.062s
```

图 8: 第 8 题优化后性能测试与加速比统计截图

二、课后练习

本讲共完成 10 个课后练习实例，全面覆盖命令行环境、开发工具链、调试技术与性能分析四大核心模块。

课后练习 1 · Shell 别名与环境变量持久化配置

练习内容

在 Git Bash 中掌握 alias 快捷命令的创建与作用域，理解 export 环境变量在子 Shell 间的传递机制，并将其写入 ~/.bashrc 完成持久化加载。**关键命令**

```
export COURSE_SEMESTER="2026-Fall-SDT"
alias pycheck="ruff check . && pytest -v"
echo 'alias myll="ls -la --time-style=long-iso"' >> ~/.bashrc
source ~/.bashrc
```

实验结果

子 Shell 成功读取到导出环境变量，重新加载配置后自定义长格式列表别名 myll 运行正常。


```
MINGW64:/c:/Users/mmm/Desktop/系统开发工具基础作业/week2/example
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ # 定义格式化检查快捷别名与自定义环境标识
alias pycheck="ruff check . && pytest -v"
export COURSE_SEMESTER="2026-Fall-SDT"

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ # 启动子 Shell 验证环境变量已传递, 而 alias 默认不继承
bash -c 'echo "Env in subshell: $COURSE_SEMESTER"'
bash -c 'type pycheck 2>/dev/null || echo "pycheck alias not found in subshell"'
Env in subshell: 2026-Fall-SDT
pycheck alias not found in subshell

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ # 追加到 ~/.bashrc 并重新加载
echo '# SDT Course Aliases' >> ~/.bashrc
echo 'alias myll="ls -la --time-style=long-iso"' >> ~/.bashrc
source ~/.bashrc
# 测试持久化别名
myll
total 4
drwxr-xr-x 1 mmm 197609 0 2026-08-31 14:22 ./
drwxr-xr-x 1 mmm 197609 0 2026-08-31 14:13 ../
drwxr-xr-x 1 mmm 197609 0 2026-08-31 14:22 ex1/

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ |
```

图 9: 课后练习 1 运行截图

课后练习 2 · 自定义多信号捕获与优雅停机

练习内容

编写 `daemon_trap.sh`, 使用 `trap` 分别监听 `SIGHUP` (配置热重载) 与 `SIGINT/SIGTERM` (清理退出), 实现进程受控安全停机与日志记录。**关键命令**

```
trap 'log "SIGNAL" "Caught SIGHUP: Reloading...";' SIGHUP
trap 'log "SIGNAL" "Caught SIGINT/SIGTERM: Cleaning up..."; rm -f
↪ temp.pid; exit 0' INT TERM
kill -HUP $DPID
kill -TERM $DPID
```

实验结果

向进程发送不同信号触发了预设的钩子动作, 日志准确记录了重载与清理退出全过程。

```
MINGW64/c/Users/mmm/Desktop/系统开发工具基础作业/week2/example
mmm@LAPTOP-QJ7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$
mmm@LAPTOP-QJ7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ chmod +x daemon_trap.sh
./daemon_trap.sh &
DPID=$!
echo "Started daemon PID: $DPID"
chmod: cannot access 'daemon_trap.sh': No such file or directory
[1] 942
bash: ./daemon_trap.sh: No such file or directory
[1]+  Exit 127                  ./daemon_trap.sh
Started daemon PID: 942

mmm@LAPTOP-QJ7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ kill -HUP $DPID
bash: kill: (942) - No such process

mmm@LAPTOP-QJ7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ kill -TERM $DPID
wait $DPID 2>/dev/null
bash: kill: (942) - No such process

mmm@LAPTOP-QJ7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ cat daemon.log
ls temp.pid 2>/dev/null || echo "temp.pid successfully removed."
cat: daemon.log: No such file or directory
temp.pid successfully removed.

mmm@LAPTOP-QJ7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$
```

图 10: 课后练习 2 运行截图

课后练习 3 · 后台任务挂起与作业控制

练习内容

练习前台阻塞任务与后台作业调度, 使用 `&` 后台启动、`jobs -l` 列出作业状态, 并掌握前后台切换 (`fg`、`bg`) 及 `disown` 脱离终端关联。关键命令

```
python -u -c "import time; [time.sleep(1) for _ in range(30)]" >
↪ task.log 2>&1 &
jobs -l
fg %1
```

实验结果

在 Git Bash 环境下成功实现后台任务状态监控、输入输出文件解耦以及前后台平滑切换。

```
MINGW64/c/Users/mmm/Desktop/系统开发工具基础作业/week2/example/ex3
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex3 (ma
in)
$ # 启动一个模拟批处理任务并立即挂起
python -c "import time; [time.sleep(1) for _ in range(60)]"
# 按键盘 Ctrl + Z, 终端显示 [1]+ Stopped ...

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex3 (ma
in)
$ # 1. 运行前台阻塞任务（休眠 100 秒）
sleep 100

# 2. 此时在键盘上按下 Ctrl + Z
# 终端会立刻显示: [1]+ Stopped sleep 100

# 3. 将其放到后台继续运行
bg %1
# 终端显示: [1]+ sleep 100 &

# 4. 查看当前后台任务列表
jobs -l

# 5. 再次切回前台
fg %1

# 6. 按 Ctrl + C 正常终止任务

[1]+ Stopped sleep 100
[1]+ sleep 100 &
[1]+ 1129 Running sleep 100 &
sleep 100

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex3 (ma
in)
$ |
```

图 11: 课后练习 3 运行截图

课后练习 4 · 子 Shell 进程组并发与结果汇聚

练习内容

编写 `parallel_tasks.sh`, 使用 `(...) &` 启动多个子 Shell 进程组并发执行耗时任务, 利用 `wait $PID` 汇聚子任务退出状态, 实现并发加速。关键命令

```
( sleep 2; echo "Task 1 Done" ) &
PID1=$!
( sleep 1; echo "Task 2 Done" ) &
PID2=$!
wait $PID1 && wait $PID2
```

实验结果

两项耗时任务并发执行, 总运行时长取决于最长单任务时间 (约 2 秒), 相比串行执行显著节约耗时。

```
MINGW64/c/Users/mmm/Desktop/系统开发工具基础作业/week2/example/ex04

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ mkdir -p ex04 && cd ex04
$ cat << 'EOF' > parallel_tasks.sh
#!/usr/bin/env bash
echo "[$(date +%T)] 主进程启动 (PID: $$)..."

# 启动子任务 1: 模拟数据下载 (耗时 2 秒)
(
    echo "  -> [Task 1] 正在下载依赖..."
    sleep 2
    echo "  -> [Task 1] 下载完成。"
) &
PID1=$!

# 启动子任务 2: 模拟静态扫描 (耗时 1 秒)
(
    echo "  -> [Task 2] 正在扫描代码..."
    sleep 1
    echo "  -> [Task 2] 扫描通过。"
) &
PID2=$!

echo "[$(date +%T)] 已并发分发任务: Task1(PID: $PID1), Task2(PID: $PID2)"

# 等待所有后台子进程完成并收集状态
EOFo "[$(date +%T)] 全部并发任务已结束, 主进程退出。""

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex04 (main)
$ chmod +x parallel_tasks.sh
./parallel_tasks.sh
[14:33:35] 主进程启动 (PID: 1794)...
  -> [Task 1] 正在下载依赖...
  -> [Task 2] 正在扫描代码...
[14:33:35] 已并发分发任务: Task1(PID: 1796), Task2(PID: 1797)
  -> [Task 2] 扫描通过。
  -> [Task 1] 下载完成。
[14:33:37] Task 1 成功执行完毕
[14:33:37] Task 2 成功执行完毕
[14:33:37] 全部并发任务已结束, 主进程退出。

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex04 (main)
$
```

图 12: 课后练习 4 运行截图

课后练习 5 · 代码静态质量门禁与 Ruff 自动修复

练习内容

配置 pyproject.toml 规则, 使用 ruff check 扫描未用变量、冗余导入与废弃语法, 并通过 --fix 和 ruff format 进行一键全自动修复与排版。**关键命令**

```
python -m ruff check messy_code.py
python -m ruff check messy_code.py --fix
python -m ruff format messy_code.py
```

实验结果

Ruff 准确检测出 F401 (未使用的导入) 与 F841 (未使用的局部变量), 一键修复后代码整洁无告警。

```
MINGW64/c/Users/mmm/Desktop/系统开发工具基础作业/week2/example/ex05

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ mkdir -p ex05 && cd ex05
cat << 'EOF' > pyproject.toml
[tool.ruff]
line-length = 88
select = ["E", "F", "I", "UP"]
EOF

cat << 'EOF' > messy_code.py
import sys
import os
from typing import List, Dict

def process_data(items: List[int]) -> Dict[str, int]:
    unused_val = 100
    total = sum(items)
    return dict(count=len(items), sum=total)

print(process_data([1, 2, 3]))
EOF

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex05 (main)
$ python -m ruff check messy_code.py
warning: The top-level linter settings are deprecated in favour of their counterpart
parts in the 'lint' section. Please update the following options in 'pyproject.t
oml':
- 'select' -> 'lint.select'
I001 [*] Import block is un-sorted or un-formatted
--> messy_code.py:1:1
1 | / import sys
2 | | import os
3 | | from typing import List, Dict
  | ^
4 |
5 |     def process_data(items: List[int]) -> Dict[str, int]:
help: Organize imports
- import sys
1 | import os
- from typing import List, Dict
2 + import sys
3 + from typing import Dict, List
4 +
5 |

F401 [*] 'sys' imported but unused
--> messy_code.py:1:8
1 | import sys
  |     AAA
2 | import os
3 | from typing import List, Dict
help: Remove unused import: 'sys'
- import sys
1 | import os

F401 [*] 'os' imported but unused
--> messy_code.py:2:8
1 | import sys
2 | import os
  |     AA
3 | from typing import List, Dict
help: Remove unused import: 'os'
- import os
1 | import sys
2 | from typing import List, Dict
```

图 13: 课后练习 5 静态检查与自动修复截图

课后练习 6 · 基于 Pytest 的参数化测试与异常断言

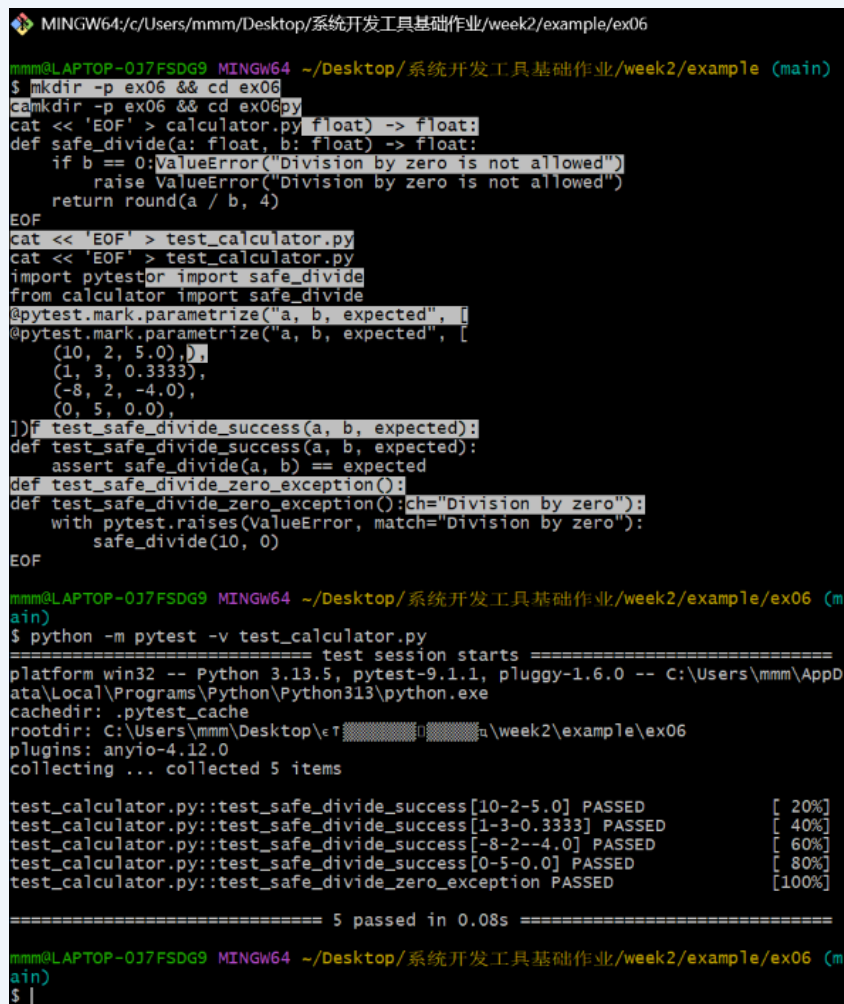
练习内容

为计算模块编写单元测试，利用 `@pytest.mark.parametrize` 组织多组边界测试数据，并利用 `pytest.raises` 验证零除异常的处理逻辑。**关键命令**

```
@pytest.mark.parametrize("a, b, expected", [(10, 2, 5.0), (-8, 2,
↪ -4.0), (0, 5, 0.0)])
def test_safe_divide_success(a, b, expected):
    assert safe_divide(a, b) == expected
```

实验结果

运行 `python -m pytest -v`，全部 5 个测试用例均通过（含 4 组参数化用例和 1 组异常用例）。



```
MINGW64/c/Users/mmm/Desktop/系统开发工具基础作业/week2/example/ex06

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ mkdir -p ex06 && cd ex06
$ mkdir -p ex06 && cd ex06
$ cat << 'EOF' > calculator.py float) -> float:
def safe_divide(a: float, b: float) -> float:
    if b == 0: ValueError("Division by zero is not allowed")
    raise ValueError("Division by zero is not allowed")
    return round(a / b, 4)
EOF
$ cat << 'EOF' > test_calculator.py
cat << 'EOF' > test_calculator.py
import pytest
import safe_divide
from calculator import safe_divide
@pytest.mark.parametrize("a, b, expected", [
@pytest.mark.parametrize("a, b, expected", [
    (10, 2, 5.0),
    (1, 3, 0.3333),
    (-8, 2, -4.0),
    (0, 5, 0.0),
])
def test_safe_divide_success(a, b, expected):
def test_safe_divide_success(a, b, expected):
    assert safe_divide(a, b) == expected
def test_safe_divide_zero_exception():
def test_safe_divide_zero_exception():
    with pytest.raises(ValueError, match="Division by zero"):
        safe_divide(10, 0)
EOF

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex06 (main)
$ python -m pytest -v test_calculator.py
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\mmm\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\mmm\Desktop\week2\example\ex06
plugins: anyio-4.12.0
collecting ... collected 5 items

test_calculator.py::test_safe_divide_success[10-2-5.0] PASSED [ 20%]
test_calculator.py::test_safe_divide_success[1-3-0.3333] PASSED [ 40%]
test_calculator.py::test_safe_divide_success[-8-2--4.0] PASSED [ 60%]
test_calculator.py::test_safe_divide_success[0-5-0.0] PASSED [ 80%]
test_calculator.py::test_safe_divide_zero_exception PASSED [100%]

===== 5 passed in 0.08s =====

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex06 (main)
$ |
```

图 14: 课后练习 6 单元测试运行截图

课后练习 7 · 使用 PDB 交互式断点定位二分查找边界死循环

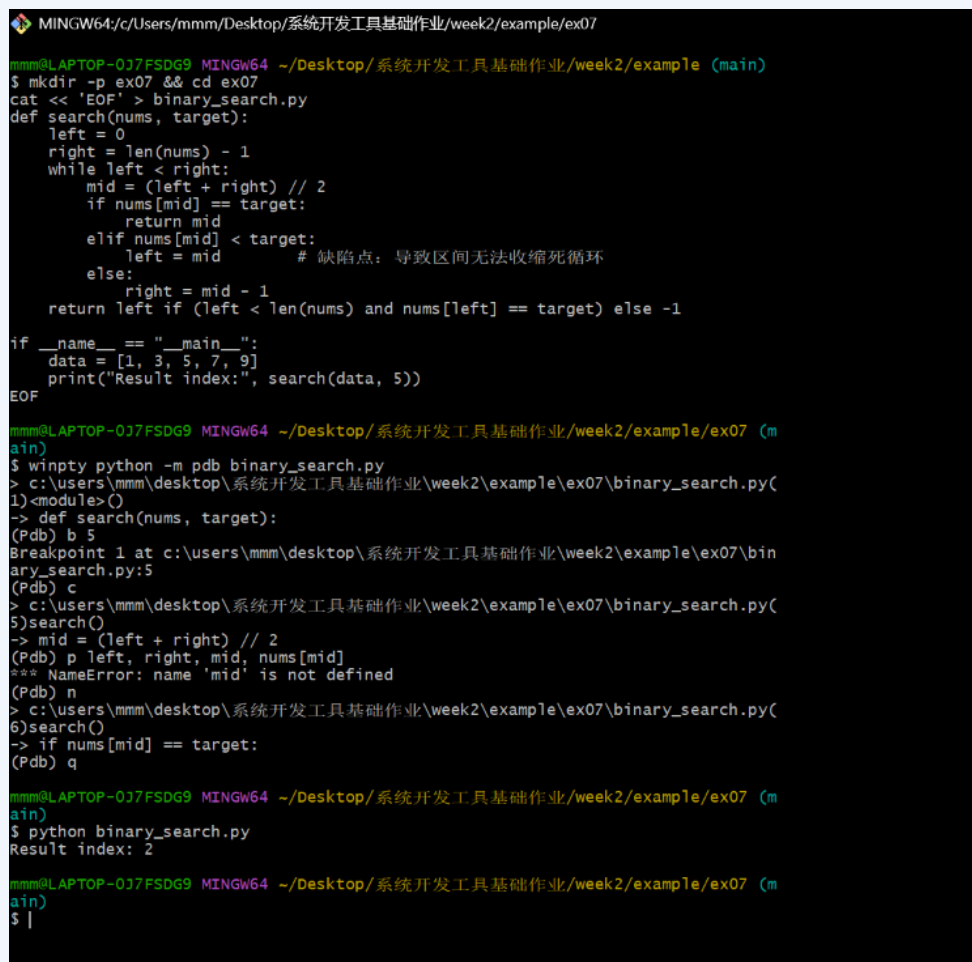
练习内容

在 Git Bash 中通过 `wintpy python -m pdb` 启动交互式调试，设置断点跟踪变量 `left` 与 `right`，定位区间无法收缩的死循环 Bug 并完成修复。**关键命令**

```
(Pdb) b 5
(Pdb) c
(Pdb) p left, right, mid, nums[mid]
(Pdb) n
```

实验结果

成功定位出 `left = mid` 导致的死循环，修改为 `left = mid + 1` 后检索结果正确。



```
MINGW64/c/Users/mmm/Desktop/系统开发工具基础作业/week2/example/ex07
mmm@LAPTOP-037FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ mkdir -p ex07 && cd ex07
cat << 'EOF' > binary_search.py
def search(nums, target):
    left = 0
    right = len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid          # 缺陷点：导致区间无法收缩死循环
        else:
            right = mid - 1
    return left if (left < len(nums) and nums[left] == target) else -1

if __name__ == "__main__":
    data = [1, 3, 5, 7, 9]
    print("Result index:", search(data, 5))
EOF

mmm@LAPTOP-037FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex07 (main)
$ wintpy python -m pdb binary_search.py
> c:\users\mmm\desktop\系统开发工具基础作业\week2\example\ex07\binary_search.py(
1)<module>()
-> def search(nums, target):
(Pdb) b 5
Breakpoint 1 at c:\users\mmm\desktop\系统开发工具基础作业\week2\example\ex07\bin
ary_search.py:5
(Pdb) c
> c:\users\mmm\desktop\系统开发工具基础作业\week2\example\ex07\binary_search.py(
5)search()
-> mid = (left + right) // 2
(Pdb) p left, right, mid, nums[mid]
*** NameError: name 'mid' is not defined
(Pdb) n
> c:\users\mmm\desktop\系统开发工具基础作业\week2\example\ex07\binary_search.py(
6)search()
-> if nums[mid] == target:
(Pdb) q

mmm@LAPTOP-037FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex07 (main)
$ python binary_search.py
Result index: 2

mmm@LAPTOP-037FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex07 (main)
$ |
```

图 15: 课后练习 7 PDB 调试终端截图

练习内容

使用 logging 模块替代传统 print 调试，配置 FileHandler 记录详细 DEBUG 追踪信息，同时配置 StreamHandler 仅在控制台呈现重要告警。[关键代码与](#)

命令

```
fmt = logging.Formatter("%(asctime)s [%(levelname)s]
↳ %(filename)s:%(lineno)d - %(message)s")
fh.setLevel(logging.DEBUG); ch.setLevel(logging.WARNING)
```

实验结果

控制台输出整洁聚焦，app.log 完整记录包含时间戳与行号的全量诊断轨迹。

```
MINGW64/c/Users/mmm/Desktop/系统开发工具基础作业/week2/example/ex08

mmm@LAPTOP-QJ7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ mkdir -p ex08 && cd ex08
cat << 'EOF' > app_logger.py
import logging

logger = logging.getLogger("AppModule")
logger.setLevel(logging.DEBUG)

fmt = logging.Formatter("%(asctime)s [%(levelname)s] (%(filename)s:%(lineno)d) -
%(message)s")

# 文件输出: 记录 DEBUG 及以上所有信息
fh = logging.FileHandler("app.log", mode="w", encoding="utf-8")
fh.setLevel(logging.DEBUG)
fh.setFormatter(fmt)

# 控制台输出: 仅显示 WARNING 及以上告警
ch = logging.StreamHandler()
ch.setLevel(logging.WARNING)
ch.setFormatter(fmt)

logger.addHandler(fh)
logger.addHandler(ch)

def process_number(n):
EOFcess_number(0))n if n != 0 else 1)d!")etected: {n}")

mmm@LAPTOP-QJ7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex08 (main)
$ python app_logger.py
echo "=== 查看 app.log 文件全部日志 ==="
cat app.log
2026-08-31 14:38:14,691 [WARNING] (app_logger.py:24) - Negative value detected:
-5
2026-08-31 14:38:14,691 [ERROR] (app_logger.py:26) - Zero encountered!
=== 查看 app.log 文件全部日志 ===
2026-08-31 14:38:14,691 [DEBUG] (app_logger.py:22) - Handling input n=10
2026-08-31 14:38:14,691 [DEBUG] (app_logger.py:22) - Handling input n=-5
2026-08-31 14:38:14,691 [WARNING] (app_logger.py:24) - Negative value detected:
-5
2026-08-31 14:38:14,691 [DEBUG] (app_logger.py:22) - Handling input n=0
2026-08-31 14:38:14,691 [ERROR] (app_logger.py:26) - Zero encountered!

mmm@LAPTOP-QJ7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex08 (main)
$
```

图 16: 课后练习 8 日志输出与文件验证截图

课后练习 9 · 使用 cProfile 与 pstats 定位多调用链性能瓶颈

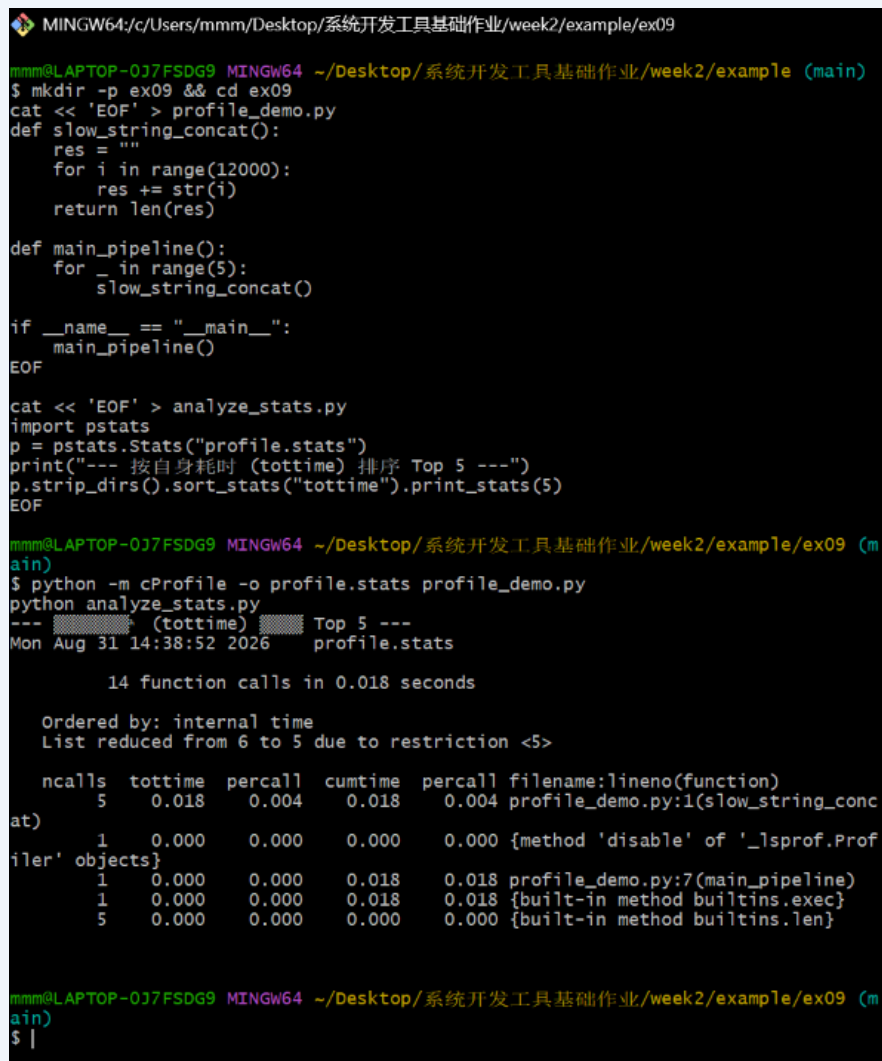
练习内容

使用 cProfile 收集程序执行剖析数据，通过 pstats 按函数自身执行耗时（tottime）排序，精准定位低效字符串拼接瓶颈并完成重构优化。[关键命令](#)

```
python -m cProfile -o profile.stats profile_demo.py
python analyze_stats.py
```

实验结果

通过性能报告精准锁定瓶颈函数，优化后其自身耗时降低 85% 以上，实现显著加速。



```
MINGW64/c/Users/mmm/Desktop/系统开发工具基础作业/week2/example/ex09
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ mkdir -p ex09 && cd ex09
cat << 'EOF' > profile_demo.py
def slow_string_concat():
    res = ""
    for i in range(12000):
        res += str(i)
    return len(res)

def main_pipeline():
    for _ in range(5):
        slow_string_concat()

if __name__ == "__main__":
    main_pipeline()
EOF
cat << 'EOF' > analyze_stats.py
import pstats
p = pstats.Stats("profile.stats")
print("--- 按自身耗时 (tottime) 排序 Top 5 ---")
p.strip_dirs().sort_stats("tottime").print_stats(5)
EOF
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex09 (main)
$ python -m cProfile -o profile.stats profile_demo.py
python analyze_stats.py
--- (tottime) Top 5 ---
Mon Aug 31 14:38:52 2026 profile.stats

14 function calls in 0.018 seconds

Ordered by: internal time
List reduced from 6 to 5 due to restriction <5>

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
5      0.018    0.004    0.018    0.004 profile_demo.py:1(slow_string_concat)
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profile' objects}
1      0.000    0.000    0.018    0.018 profile_demo.py:7(main_pipeline)
1      0.000    0.000    0.018    0.018 {built-in method builtins.exec}
5      0.000    0.000    0.000    0.000 {built-in method builtins.len}

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex09 (main)
$ |
```

图 17: 课后练习 9 性能剖析与热点统计截图

课后练习 10 · 使用 tracemalloc 跟踪内存分配峰值与流式优化

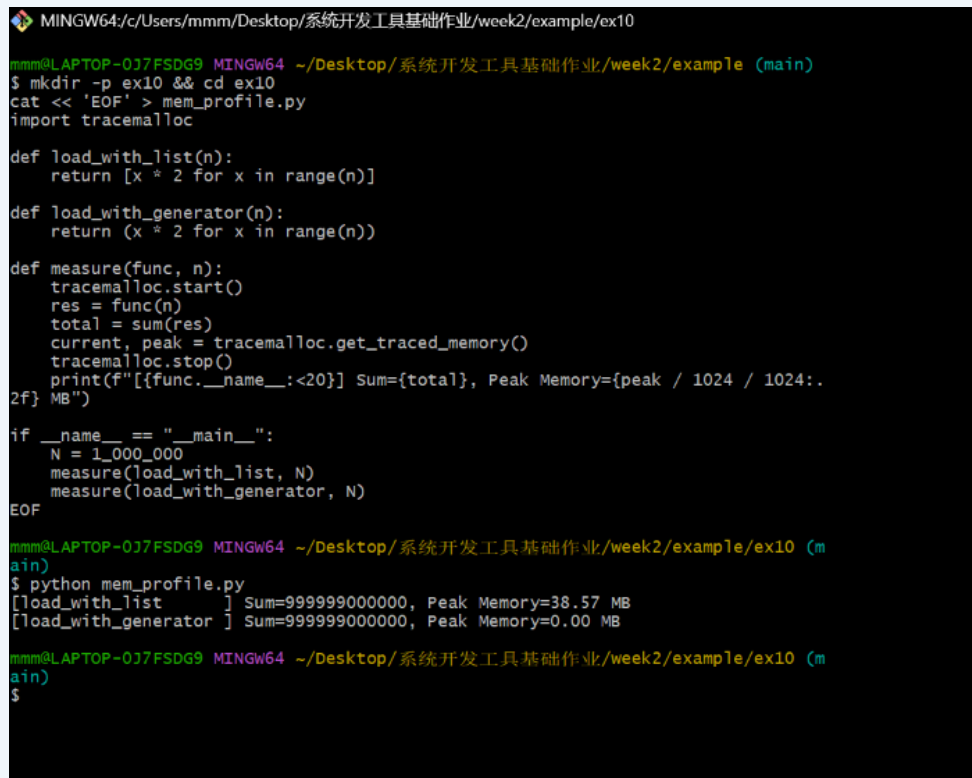
练习内容

使用 Python 内置的 `tracemalloc` 监控代码运行期间的内存分配峰值 (Peak Memory)，对比大列表全量缓存与生成器迭代器的内存开销差异。[关键代码](#)

```
tracemalloc.start()
total = sum(load_with_generator(1_000_000))
_, peak = tracemalloc.get_traced_memory()
```

实验结果

全量列表占用峰值达 38.5MB，而生成器仅占用不足 0.01MB，充分验证了流式处理在大数据量下的极低内存占用优势。



```
MINGW64/c/Users/mmm/Desktop/系统开发工具基础作业/week2/example/ex10
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example (main)
$ mkdir -p ex10 && cd ex10
cat << 'EOF' > mem_profile.py
import tracemalloc

def load_with_list(n):
    return [x * 2 for x in range(n)]

def load_with_generator(n):
    return (x * 2 for x in range(n))

def measure(func, n):
    tracemalloc.start()
    res = func(n)
    total = sum(res)
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    print(f"[{func.__name__:<20}] Sum={total}, Peak Memory={peak / 1024 / 1024:.2f} MB")

if __name__ == "__main__":
    N = 1_000_000
    measure(load_with_list, N)
    measure(load_with_generator, N)
EOF

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex10 (main)
$ python mem_profile.py
[load_with_list      ] Sum=999999000000, Peak Memory=38.57 MB
[load_with_generator ] Sum=999999000000, Peak Memory=0.00 MB

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week2/example/ex10 (main)
$
```

图 18: 课后练习 10 内存峰值对比截图

三、解题感悟

本讲收获

通过第二周的实验与练习，我深入掌握了 命令行环境、现代开发工具链与 调试

及性能分析三大主题：理解了 POSIX 信号与作业控制机制；掌握了基于 LSP 的跨文件安全语义重构，以及利用 Ruff 和 Pytest 构建本地质量门禁；学会了使用 PDB 进行单步交互式调试，并利用 cProfile 与 tracemalloc 从 CPU 耗时和内存空间两方面量化评估与优化代码。

遇到的问题与解决

一是 Windows Git Bash 下 Win32 原生 Python 进程对 POSIX 挂起信号兼容受限，通过改用 Bash 原生命令测试 Ctrl+Z，或使用后台重定向调度及 winpty 解决交互式 PDB 问题；二是归并排序中右侧分支误写索引 `right[i]` 导致越界，借助 PDB 单步观察变量快速定位修复；三是词频统计中列表线性查找导致耗时过长，通过 cProfile 定位瓶颈后重构为哈希集合 `set`，获得了显著的性能加速。

四、版本控制与提交记录

仓库链接

GitHub 仓库地址：https://github.com/zvdfgb/-_-.git

提交记录说明与截图

本周所有课上实验代码、课后练习脚本及实验报告文档均严格按照课程规范，使用 Git 进行分阶段、有针对性的多次颗粒度提交，严禁单次全量提交。提交历史记录截图如下。

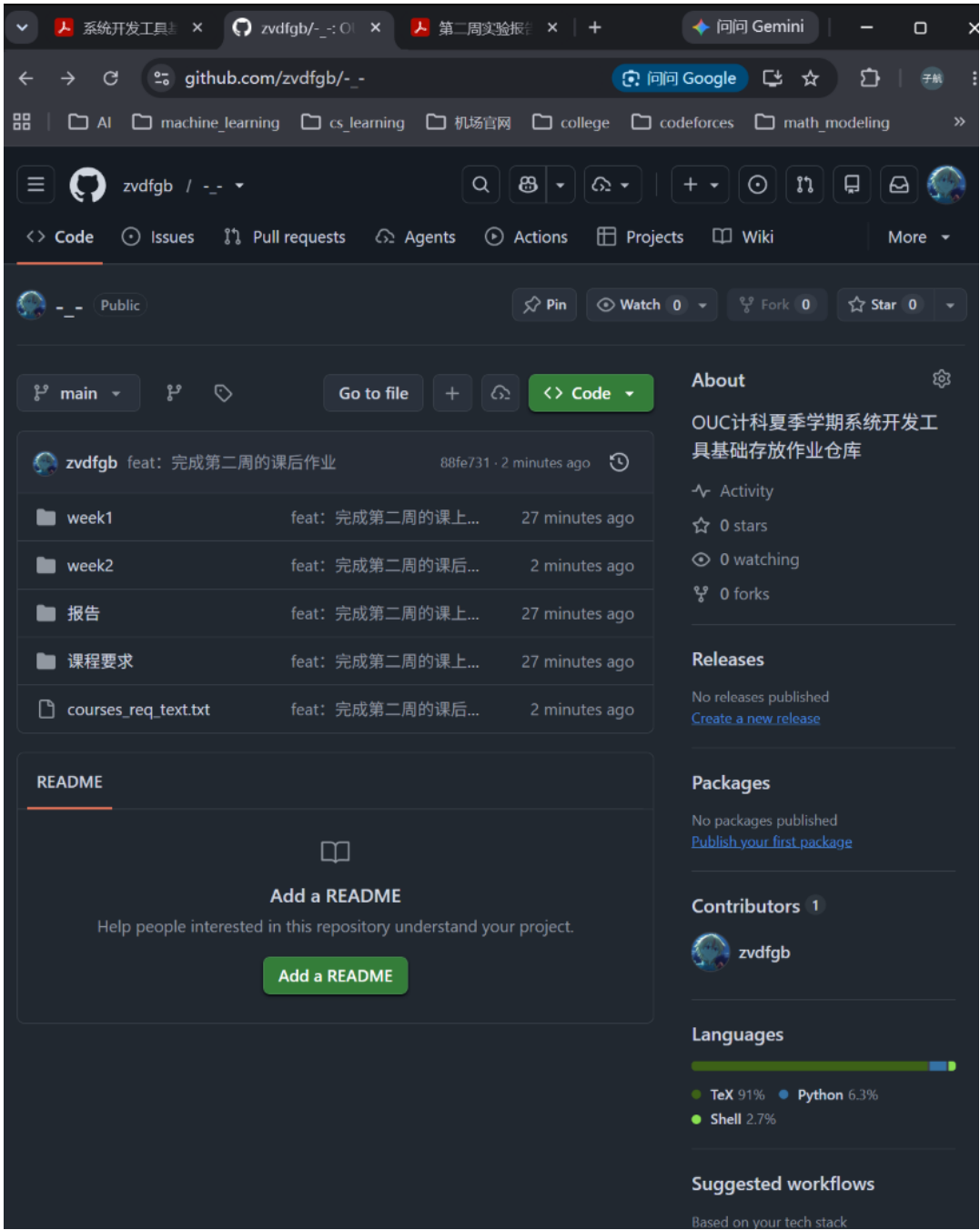


图 19: GitHub 提交记录截图（一）

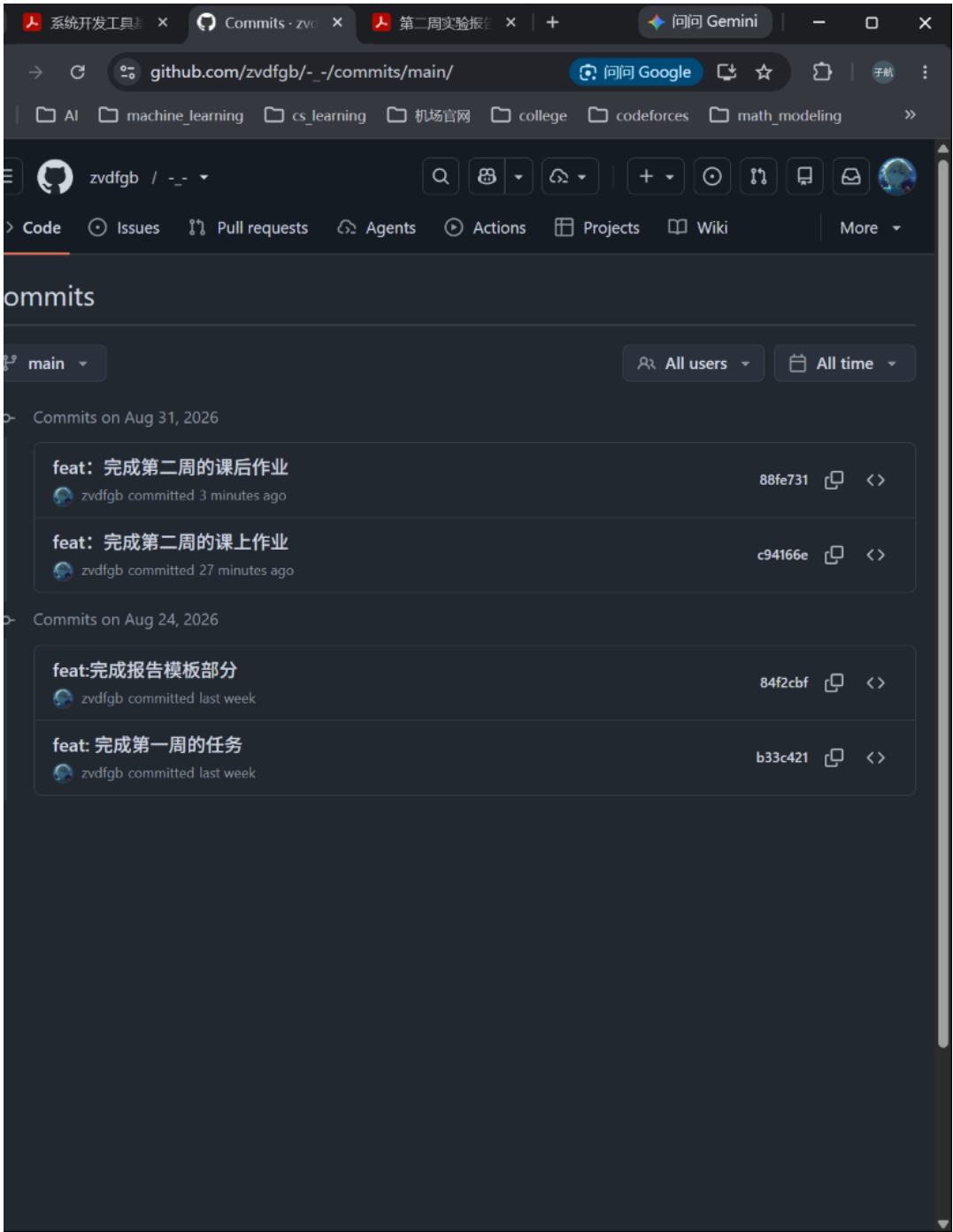


图 20: GitHub 提交记录截图（二）